Engineering Portfolio — Prerana Kapadnis

Full-Stack Engineer & AI Builder | MERN, PERN, & AI/ML

Formally designing high-throughput data pipelines and fine-tuning LLMs. Informally here to experiment, try things that shouldn't work, and see what happens when you push tech to its limits.

📬 **Connect:** [LinkedIn](#) | [GitHub](#) | Email

🎯 Overview

This notebook is a living, executable showcase of my core engineering milestones, architecture choices, and experimental projects. **Run the cells below** to see live pipeline simulations, optimization metrics, and model evaluations.

[Initialization] Run this cell to boot up Prerana's Engineering Portfolio Dashboard

```python
# @title [Initialization] Run this cell to boot up Prerana's Engineering Portfolio Dashboard { display-mode: "form" }
import time
import sys
import pandas as pd
import numpy as np
import plotly.graph_objects as go
import plotly.express as px
import ipywidgets as widgets
from IPython.display import display, HTML, clear_output

print("🚀 Initializing Portfolio Core Engine...")
time.sleep(0.5)
print(f"🖥 System Python Version: {sys.version.split()[0]}")
print("📦 Core Modules Loaded: Pandas, Plotly, IPyWidgets")
print("✅ Status: Ready to Explore.")
```

```
🚀 Initializing Portfolio Core Engine...
🖥 System Python Version: 3.12.13
📦 Core Modules Loaded: Pandas, Plotly, IPyWidgets
✅ Status: Ready to Explore.
```


[Initialization] Run this cell to boot up Prerana's Engineering Portfolio Dashboard

```python
# @title [Initialization] Run this cell to boot up Prerana's Engineering Portfolio Dashboard { display-mode: "form" }
import time
import sys
import pandas as pd
import numpy as np
import plotly.graph_objects as go
import plotly.express as px
import ipywidgets as widgets
from IPython.display import display, HTML, clear_output

print("🚀 Initializing Portfolio Core Engine...")
time.sleep(0.5)
print(f"🖥 System Python Version: {sys.version.split()[0]}")
print("📦 Core Modules Loaded: Pandas, Plotly, IPyWidgets")
print("✅ Status: Ready to Explore.")
```

```
🚀 Initializing Portfolio Core Engine...
🖥 System Python Version: 3.12.13
📦 Core Modules Loaded: Pandas, Plotly, IPyWidgets
✅ Status: Ready to Explore.
```

🔍 Interactive Project Filter

```python
# @title 🔍 Interactive Project Filter { display-mode: "form" }

# Data array representing your deep-dive execution history
projects_db = [
    {"name": "Genesis (Autonomize.AI)", "domain": "Fullstack / Data Pipelines", "tech": "PERN, FastAPI, Apache Kafka", "impact": "Ingested streaming data with real-time JSON p
    {"name": "Capgemini UI Frameworks", "domain": "Fullstack / Data Pipelines", "tech": "JavaScript, HTML/CSS, Docker", "impact": "Enhanced internal web app UI, supported Agil
    {"name": "Bash Scripting Assistant", "domain": "AI / ML", "tech": "QLoRA, CodeLlama-7B, Phi-3, Streamlit", "impact": "Fine-tuned CodeLlama-7B on custom NL-to-bash dataset.
    {"name": "AI-Driven B2B Pharmacy System", "domain": "AI / ML", "tech": "Python, Flask, PostgreSQL", "impact": "Parallelized model training achieving a 30% reduction in run
    {"name": "Network Traffic Analyzer", "domain": "Low-Level & Logic", "tech": "C++, Wireshark, Socket Programming", "impact": "Built a multithreaded network packet analysis
    {"name": "Automated Grading System", "domain": "Backend Engine", "tech": "Java, Spring Boot, Hibernate, MySQL", "impact": "Engineered scalable backend achieving over 95% t
]

dropdown = widgets.Dropdown(
    options=['All Domains', 'Fullstack / Data Pipelines', 'AI / ML', 'Low-Level & Logic', 'Backend Engine'],
    value='All Domains',
    description='Domain:',
    disabled=False,
)

def display_filtered_projects(change):
    clear_output(wait=True)
    display(dropdown)
    selected = change['new'] if change else 'All Domains'

    print(f"\n--- Showing Projects for: {selected} ---\n")
    for p in projects_db:
        if selected == 'All Domains' or p['domain'] == selected:
            html_box = f"""
            <div style="border-left: 4px solid #1a73e8; padding: 10px; margin-bottom: 15px; background-color: #f8f9fa; border-radius: 0 4px 4px 0;">
                <b style="font-size: 16px; color: #202124;">{p['name']}</b> <span style="color: #5f6368; font-size: 12px;">({p['tech']})</span>
                <p style="margin: 5px 0 0 0; color: #3c4043; font-size: 14px;"><b>Impact Execution:</b> {p['impact']}</p>
            </div>
            """
            display(HTML(html_box))

dropdown.observe(display_filtered_projects, names='value')
display(dropdown)
display_filtered_projects(None)
```

Domain: Backend Engine

Domain: All Domains

```
--- Showing Projects for: All Domains ---
```

Genesis (Autonomize.AI) (PERN, FastAPI, Apache Kafka)
Impact Execution: Ingested streaming data with real-time JSON parsing and built a robust document validation pipeline resolving critical webhooks.

Capgemini UI Frameworks (JavaScript, HTML/CSS, Docker)
Impact Execution: Enhanced internal web app UI, supported Agile sprints via feature dev, and implemented JUnit testing.

Bash Scripting Assistant (QLoRA, CodeLlama-7B, Phi-3, Streamlit)
Impact Execution: Fine-tuned CodeLlama-7B on custom NL-to-bash dataset. Achieved 0.9 CodeBLEU and 0.8 ROUGE alignment.

AI-Driven B2B Pharmacy System (Python, Flask, PostgreSQL)
Impact Execution: Parallelized model training achieving a 30% reduction in runtime. Published research paper.

Network Traffic Analyzer (C++, Wireshark, Socket Programming)
Impact Execution: Built a multithreaded network packet analysis tool capturing, filtering, and diagnosing TCP/UDP traffic.

Automated Grading System (Java, Spring Boot, Hibernate, MySQL)
Impact Execution: Engineered scalable backend achieving over 95% test coverage using JUnit/Mockito. Reduced query response times by 40%.

Domain: Fullstack / Data Pipelines

```
--- Showing Projects for: Fullstack / Data Pipelines ---
```

Genesis (Autonomize.AI) (PERN, FastAPI, Apache Kafka)
Impact Execution: Ingested streaming data with real-time JSON parsing and built a robust document validation pipeline resolving critical webhooks.

Capgemini UI Frameworks (JavaScript, HTML/CSS, Docker)
Impact Execution: Enhanced internal web app UI, supported Agile sprints via feature dev, and implemented JUnit testing.

Domain: Fullstack / Data Pipelines

--- Showing Projects for: Fullstack / Data Pipelines ---

Genesis (Autonomize.AI) (PERN, FastAPI, Apache Kafka)
Impact Execution: Ingested streaming data with real-time JSON parsing and built a robust document validation pipeline resolving critical webhooks.

Capgemini UI Frameworks (JavaScript, HTML/CSS, Docker)
Impact Execution: Enhanced internal web app UI, supported Agile sprints via feature dev, and implemented JUnit testing.

Domain: AI / ML

```
--- Showing Projects for: AI / ML ---
```

Bash Scripting Assistant (QLoRA, CodeLlama-7B, Phi-3, Streamlit)
Impact Execution: Fine-tuned CodeLlama-7B on custom NL-to-bash dataset. Achieved 0.9 CodeBLEU and 0.8 ROUGE alignment.

AI-Driven B2B Pharmacy System (Python, Flask, PostgreSQL)
Impact Execution: Parallelized model training achieving a 30% reduction in runtime. Published research paper.

Domain: Low-Level & Logic

```
--- Showing Projects for: Low-Level & Logic ---
```

Network Traffic Analyzer (C++, Wireshark, Socket Programming)
Impact Execution: Built a multithreaded network packet analysis tool capturing, filtering, and diagnosing TCP/UDP traffic.

Domain: Backend Engine

```
--- Showing Projects for: Backend Engine ---
```

Automated Grading System (Java, Spring Boot, Hibernate, MySQL)
Impact Execution: Engineered scalable backend achieving over 95% test coverage using JUnit/Mockito. Reduced query response times by 40%.

🧪 Deep Tech & AI/ML Projects

🤖 Bash Scripting Assistant (Fine-Tuning CodeLlama)

- [cite_start]**Core Work:** Fine-tuned CodeLlama-7B on a custom natural language-to-bash dataset using **QLoRA** for efficient memory optimization[cite: 36, 38].
- [cite_start]**Results:** Achieved a **0.9 CodeBLEU** and **0.8 ROUGE** score, proving immense semantic alignment[cite: 39].

💊 AI-Driven B2B Pharmacy Ordering System

- [cite_start]**Core Work:** Designed a modular backend architecture using Flask and PostgreSQL[cite: 33]. [cite_start]Parallelized model training to achieve a **30% reduction in training runtime**[cite: 35].

- [cite_start]**Publication:** Published a formal research paper on this framework[cite: 35].

📊 Optimization & Impact Metrics

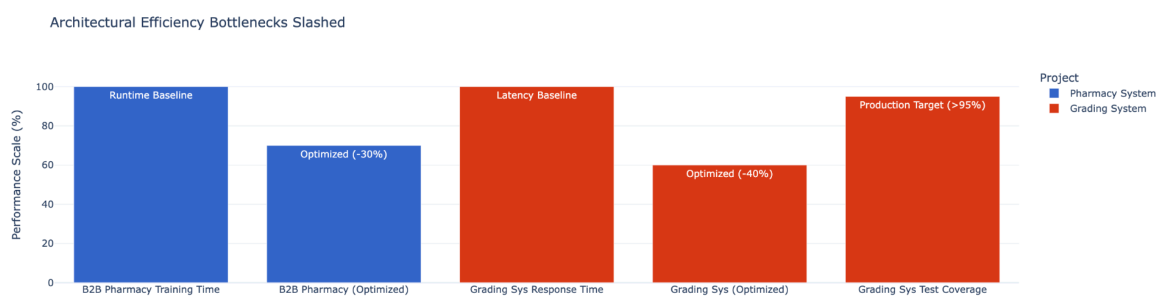
```python
# @title 📊 Optimization & Impact Metrics { display-mode: "code" }

# Data metrics pulled directly from your architectural wins
metrics_data = {
    'Metric Showcase': [
        'B2B Pharmacy Training Time', 'B2B Pharmacy (Optimized)',
        'Grading Sys Response Time', 'Grading Sys (Optimized)',
        'Grading Sys Test Coverage'
    ],
    'Value': [100, 70, 100, 60, 95], # Representing percentages/baselines
    'Type': ['Runtime Baseline', 'Optimized (-30%)', 'Latency Baseline', 'Optimized (-40%)', 'Production Target (>95%)'],
    'Project': ['Pharmacy System', 'Pharmacy System', 'Grading System', 'Grading System', 'Grading System']
}

df = pd.DataFrame(metrics_data)

fig = px.bar(
    df,
    x='Metric Showcase',
    y='Value',
    color='Project',
    text='Type',
    title="Architectural Efficiency Bottlenecks Slashed",
    labels={'Value': 'Performance Scale (%)'},
    color_discrete_sequence=px.colors.qualitative.G10
)

fig.update_traces(textposition='inside', textfont_size=12)
fig.update_layout(
    template='plotly_white',
    height=450,
    xaxis_title="",
    yaxis_range=[0, 110]
)
fig.show()
```



🏗️ Core Engineering Mentality: Stream Pipelines

[9]
0s

```python
# @title 🏗️ Core Engineering Mentality: Stream Pipelines { display-mode: "form" }

pipeline_step = widgets.SelectionSlider(
    options=['1. Ingestion', '2. Stream Processing', '3. Validation Layer', '4. High-Availability Persistence'],
    value='1. Ingestion',
    description='Pipeline Stage:',
    disabled=False,
    continuous_update=False,
    orientation='horizontal',
    readout=True
)

def explain_architecture(change):
    clear_output(wait=True)
    display(pipeline_step)
    stage = change['new'] if change else '1. Ingestion'

    explanations = {
        '1. Ingestion': "📥 <b>Ingestion Stack:</b> Utilizing PERN stacks and FastAPI to capture enterprise document workflows cleanly at the front gates.",
        '2. Stream Processing': "⚙️ <b>Stream Processing:</b> Orchestrating real-time <b>Apache Kafka</b> pipelines to handle live ingestion, giving granular visibility into JS
        '3. Validation Layer': "🛡️ <b>Validation & Fallbacks:</b> Isolating and intercepting field-level mismatches using automated validation pipelines to securely route webho
        '4. High-Availability Persistence': "💾 <b>Persistence:</b> Mapping structured JSON histories cleanly into PostgreSQL/MongoDB clusters with full backward compatibility
    }

    display(HTML(f"""
    <div style="margin-top: 20px; padding: 15px; border: 1px dashed #34a853; background-color: #f1f8e9; border-radius: 4px;">
        <p style="font-size: 15px; color: #1b5e20; font-family: monospace;">{explanations[stage]}</p>
    </div>
    """))

pipeline_step.observe(explain_architecture, names='value')
display(pipeline_step)
explain_architecture(None)
```

Pipeline St... 2. Stream Proce

⚙️ **Stream Processing:** Orchestrating real-time **Apache Kafka** pipelines to handle live ingestion, giving granular visibility into JSON payload structures.